

EasySave v1.0

Technical Documentation

1. Overview

EasySave is a backup automation tool developed as part of the ProSoft software suite. It lets users define up to five backup jobs, each capable of performing a full or differential copy of a directory — whether it sits on a local disk, an external drive, or a network share.

The software ships as a .NET 10.0 console application and relies on a standalone logging library, EasyLog.dll, that other ProSoft products can reuse without modification.

2. Architecture

2.1 Projects

Project	Type	Purpose
EasySave	Console application (.exe)	Interactive menu, CLI mode, job management
EasyLog	Class library (.dll)	Daily log file and real-time state file

2.2 EasySave Class Overview

File	Role
Program.cs	Entry point. Handles the interactive menu and the CLI argument path.
BackupManager.cs	Runs jobs. Picks the right strategy and delegates execution.
Models/BackupJob.cs	Holds the four fields that describe a job: name, source, target, type.
Models/BackupType.cs	Enum with two values: Full and Differential.
Services/IBackupStrategy.cs	Interface that both strategy classes implement.
Services/FullBackupStrategy.cs	Copies every file in the source tree, overwriting the target each time.
Services/DifferentialBackupStrategy.cs	Transfers only files missing or more recent than their target counterpart.
Config/ConfigManager.cs	Reads and writes config.json in the user's AppData folder.
Languages/LanguageManager.cs	Holds English, French and Spanish string dictionaries.

2.3 EasyLog Class Overview

File	Role
LogEntry.cs	Six-field model for a single file-transfer record.
StateEntry.cs	Eight-field model for the live state of one job.
DailyLogger.cs	Reads the current day file, appends the new entry, writes it back.
StateLogger.cs	Keeps an in-memory list of all job states and rewrites state.json after every file.

2.4 Strategy Pattern

BackupManager holds a reference to IBackupStrategy and picks a concrete implementation at runtime based on the job type. Adding a third backup mode in a future version only requires writing a new class that implements the interface — no changes to existing code.

3. Features

3.1 Backup Jobs

- Up to five named jobs can be configured and persist across restarts.
- Each job stores a name, a source directory, a target directory, and a type.
- Source and target can be local disks, external drives, or network shares (UNC paths).

3.2 Backup Types

Type	Behaviour
Full	Every file in the source is copied to the target on every run, overwriting whatever is already there.
Differential	Only files absent in the target or whose source modification date is newer are transferred.

3.3 Command-Line Mode

- EasySave.exe 1-3 → runs jobs 1, 2 and 3 sequentially
- EasySave.exe 1;3 → runs jobs 1 and 3

3.4 Language Support

The application supports English, French and Spanish. The chosen language is saved to lang.txt in the AppData folder and applied on every subsequent launch.

4. Log Files

4.1 Daily Log — YYYY-MM-DD.json

Location: %APPDATA%\EasySave\Logs\YYYY-MM-DD.json

Field	Type	Description
Name	string	Name of the backup job that triggered the transfer.
FileSource	string	Full path of the source file (UNC-compatible).
FileTarget	string	Full path of the destination file.
FileSize	long	File size in bytes at the time of transfer.
FileTransferTime	double	Duration in milliseconds. Negative value = copy error.
Time	string	Timestamp formatted as dd/MM/yyyy HH:mm:ss.

4.2 State File — state.json

Location: %APPDATA%\EasySave\State\state.json

Field	Type	Description
Name	string	Job name, matches config.json.
State	string	One of: ACTIVE, END.
SourceFilePath	string	Path of the file currently being copied.
TargetFilePath	string	Destination path of the file currently being copied.
TotalFilesToCopy	int	Number of files selected for this run.
TotalFileSize	long	Combined size in bytes of all selected files.
NbFilesLeftToDo	int	Files remaining to copy.
Progression	int	Completion percentage (0 to 100).

5. Configuration & File Locations

File	Path
Job configuration	%APPDATA%\EasySave\config.json
Daily log	%APPDATA%\EasySave\Logs\YYYY-MM-DD.json
State file	%APPDATA%\EasySave\State\state.json
Language setting	%APPDATA%\EasySave\lang.txt

Using AppData ensures the path works on any customer server without requiring administrator rights or assumptions about drive letters. Paths like C:\temp are deliberately avoided.

6. Minimum Requirements

Component	Requirement
Operating system	Windows 10 / Windows Server 2016 or later
Runtime	.NET 10.0 Runtime
Disk space	10 MB for the application + sufficient space on the target drive
Permissions	Read access on source directory, write access on target directory